RICE UNIVERSITY

Data Parallelism

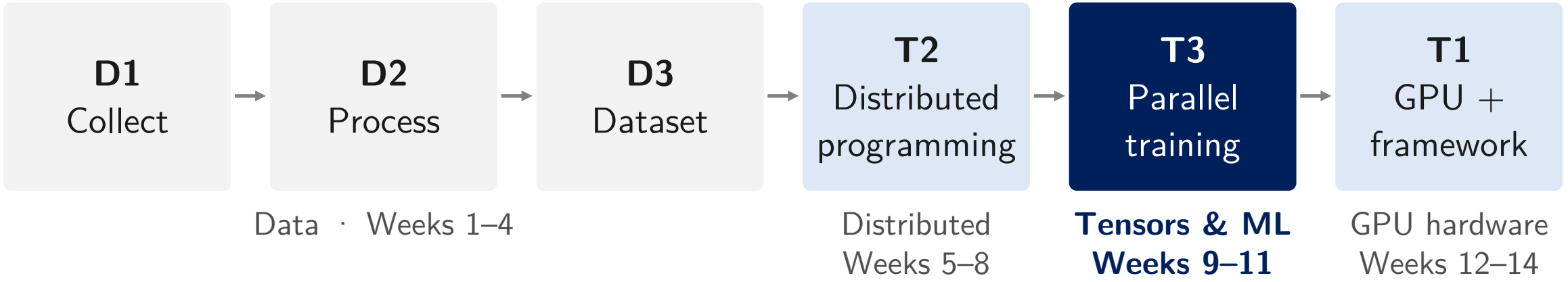
Many copies of one model, one averaged gradient, and what the copies cost in memory

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 9, Lecture 1

Where we are in the pipeline



In our pipeline

Today: train **one model on many GPUs** by splitting the batch. This week: **T2 checkpoint** and **Lab 4 due**; T3 is released after Week 10, Lecture 2.

Week 8: collectives and their cost → today: the one all-reduce a training step needs

Why train on many GPUs?

Train a 7.5-billion-parameter model on 2 trillion tokens.
You have **one H100 GPU**.

How long does training take?

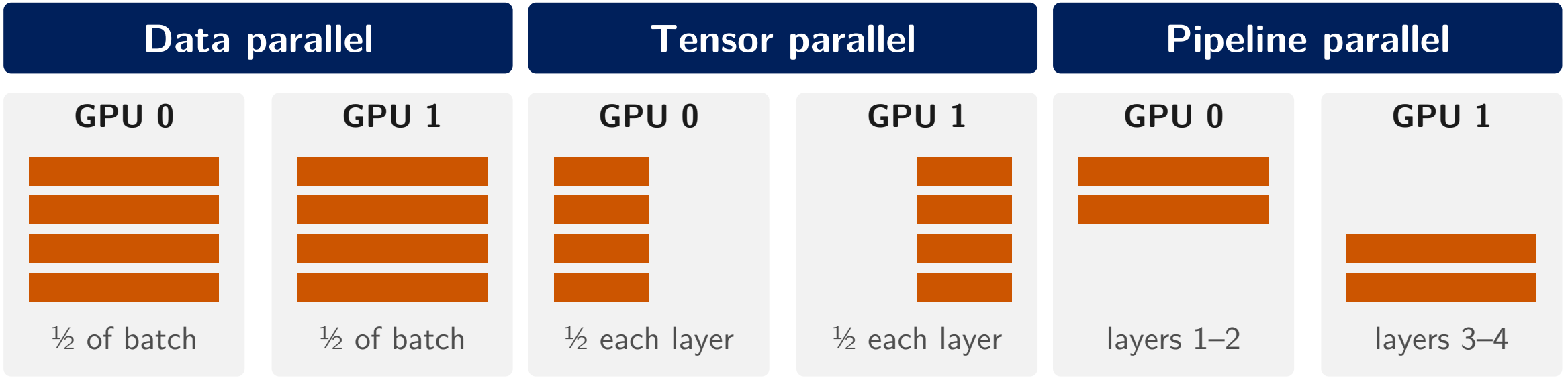


Answer

One GPU is **far too slow**. We must split **one training run** across many GPUs — but how do we split it?

Source: Estimate: 6 FLOP/parameter/token (Kaplan et al. 2020); 2T tokens (Llama 2); H100 bf16 peak (datasheet) at an assumed 40%.

Three ways to split training



copy model, split batch

today

split every layer

Week 10, Lecture 1

give each GPU some layers

Week 10, Lecture 2

Key idea

Data parallelism comes first: the model is **copied, not cut**, so the model code does not change — only one averaged gradient per step.

Where data parallelism shows up

- **PyTorch**: **DDP** trains one model on many GPUs; **FSDP** also saves memory.
- **Large models**: the outer split, combined with tensor and pipeline parallelism.
- **Costs**: one all-reduce **per step**, and a full copy of the model on **every** GPU.

In our pipeline

T3, part 1: your data-parallel loop vs a single-process reference vs DDP, on D3 data. Lab 5: tensor and pipeline.

Why data parallelism?

One GPU would take **years**. Data parallelism is the first split: **the same model on more data per step**, with one averaged gradient.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Learning goals for today

- **Explain** why averaging the GPUs' gradients gives the full-batch gradient — and fix it when shards are unequal.
- **Estimate** the one all-reduce per step, and explain how **DDP** hides it behind backward.
- **Account** for the memory of a step: **16 bytes per parameter** plus the **activations**.
- **Predict** what adding GPUs does to per-GPU memory: which part shrinks, which does not.

A puzzle to start

One GPU
global batch of 64 examples
uses **72 GB** per step

8 GPUs, data parallel
same global batch of 64
8 examples per GPU

How much memory does each of the 8 GPUs use?

Answer

≈ **30 GB, not 9 GB.** Every GPU keeps a full copy of the model and optimizer (24 GB); only the batch's share is divided.

Source: Assumed round numbers for a 1.5-billion-parameter model; Part 3 explains the split.

Recap: the all-reduce

- **all-reduce**: each GPU starts with its own vector; every GPU ends with the **sum**.
- $=$ **reduce-scatter** (GPU k gets the sum of slice k) $+$ **all-gather** (everyone collects all slices).

$$T(S, N) = 2(N-1)\,\alpha + 2(N-1)/N \cdot S/\beta$$

Week 8

α: latency per step, β: bandwidth. Large S: **T** $\approx$ **2S**/β. Small S: the **latency** term wins.

Part 1

Data parallelism

Why is averaging gradients the right thing to do?

Replicate the model, split the batch

global batch, B = 64 examples

shard 0 · 16 examples	shard 1 · 16 examples	shard 2 · 16 examples	shard 3 · 16 examples
↓	↓	↓	↓
GPU 0 · θ forward + backward	**GPU 1** · θ forward + backward	**GPU 2** · θ forward + backward	**GPU 3** · θ forward + backward
g[0]	g[1]	g[2]	g[3]

all-reduce: every GPU gets (g[0] + g[1] + g[2] + g[3]) / 4

every GPU: $\theta \leftarrow$ optimizer step(θ, averaged gradient) $\rightarrow$ copies stay identical

The gradient of a mean is a mean

$$L(\theta) = (1/B) \sum \ell(x; \theta) \qquad \text{sum over the global batch}$$

$$g[k] = (1/b) \sum \nabla \ell(x; \theta) \qquad \text{sum over shard } k\text{, size } b$$

$$\nabla L = (1/B) \sum b \cdot g[k] \qquad \text{group the sum by shard}$$

$$b = B \,/\, N \;\Rightarrow\; \nabla L = (1/N) \sum g[k]$$

Result

Equal shards: **all-reduce SUM, then divide by N**. Exact, not an approximation.

Unequal shards: weight by shard size

$$\nabla L = \Sigma\ (b[k]\ /\ B) \cdot g[k], \qquad B = \Sigma\ b[k]$$

Worker 0
9 examples $\rightarrow$ weight 9/17

Worker 1
8 examples $\rightarrow$ weight 8/17

B = 17 (T3's test)

Watch out

The plain average is **wrong** here. Fix: each rank sends **b[k]** · **g[k]**, all-reduce SUM, divide by **B**.

Sum or mean? Know your loss

Loss on each rank	After all-reduce SUM, divide by
mean over its b examples (equal shards)	N (number of ranks)
sum over its examples	B (global batch)

Watch out

A missing or extra factor is a **silent learning-rate change**. Write the single-process loss first, then match it.

Per-GPU batch, global batch

$$B = N \cdot b$$

- **b**: examples per GPU per step. **B**: examples per optimizer step.
- Fix **B**, change N: same gradient, **same** learning rate.
- Fix **b**, add GPUs: B grows — a **different** problem; retune the learning rate.
- Our MLP: B = 64 on N = 4 GPUs, so b = 16.

Data parallelism by hand

```
for x, y in loader:              # local shard
    opt.zero_grad()
    loss = F.cross_entropy(model(x), y)
    loss.backward()              # local g[k]
    for p in model.parameters():
        dist.all_reduce(p.grad) # SUM, in place
        p.grad /= world_size     # average
    opt.step()                   # same on all
```

- Only **three** new lines.
- One all-reduce per tensor.

Watch out

All ranks must call collectives in the **same order** with the **same shapes**, or the job hangs.

Source: PyTorch documentation, torch.distributed.all_reduce (default op = ReduceOp.SUM, in place).

Close, but not bit-for-bit

Cause	Effect	What to do
Floating-point sum order	last bits differ	compare with a tolerance
Dropout	different random masks	turn it off for the check
Sampler padding	repeated examples	check shards cover B once

In our pipeline

T3 compares DP with a single-process reference **within a tolerance** — know why the tolerance is needed.

Check yourself

Two minutes with a neighbour.

- Why may we average per-GPU gradients at all? State the condition.
- Two ranks get 30 and 34 of a 64-example batch. What should each rank send?
- Ranks use a **sum** loss, but you all-reduce and divide by N. By what factor is the gradient off ($B = 64$, $N = 4$)?

Part 2

The cost, and how DDP hides it

How long does agreeing on a gradient take?

Cost for our MLP

$$\mathcal{S} = 34{,}338 \times 4 \text{ bytes} = 137{,}352 \text{ bytes}$$

$$\text{bandwidth term: } \; 2 \cdot 3/4 \cdot \mathcal{S} \,/\, \beta \;\approx\; 0.46\ \mu\text{s}$$

$$\text{latency term: } \; 2(\mathcal{N} - 1)\ \alpha \;\approx\; 30\ \mu\text{s}$$

Memory

Our all-reduce is **latency-bound**. Fewer, larger messages help; more bandwidth does not.

Source: H100 datasheet (NVLink 900 GB/s bidirectional); $\alpha \approx 5$ µs assumed — use your T2 fit.

Meet a bigger model

Our MLP
$\Psi = 34{,}338$

A large language model
$\Psi = 7.5 \times 10^9$

Ψ = number of parameters. Today a model is just Ψ numbers.

$$\text{bf16 gradient} = 2 \text{ bytes} \times \Psi = 15 \text{ GB}$$

Definition

bf16: a 2-byte number format GPUs compute with fast. Bytes = $\Psi \times$ bytes per number.

Cost for the 7.5-billion-parameter model

$$\mathcal{S} = 15 \text{ GB of bf16 gradients}$$

Where the GPUs are	All-reduce time
8 GPUs in one server (NVLink)	$\approx$ 58 ms
64 GPUs in 8 servers (network)	$\approx$ 0.59 s

Memory

Large models are **bandwidth-bound**: $T \approx 2S/\beta$ per step, and crossing servers costs $\approx 10\times$.

Source: NVIDIA DGX H100 User Guide (8× ConnectX-7, 400 Gb/s per GPU); H100 datasheet (NVLink 900 GB/s bidirectional).

DDP: the same loop, wrapped

```
dist.init_process_group("nccl")
torch.cuda.set_device(local_rank)  # Week 7
model = DDP(MLP().cuda(), device_ids=[local_rank])
opt = torch.optim.AdamW(model.parameters())
for x, y in loader:       # this rank's shard
    opt.zero_grad()
    F.cross_entropy(model(x), y).backward()
    opt.step()
```

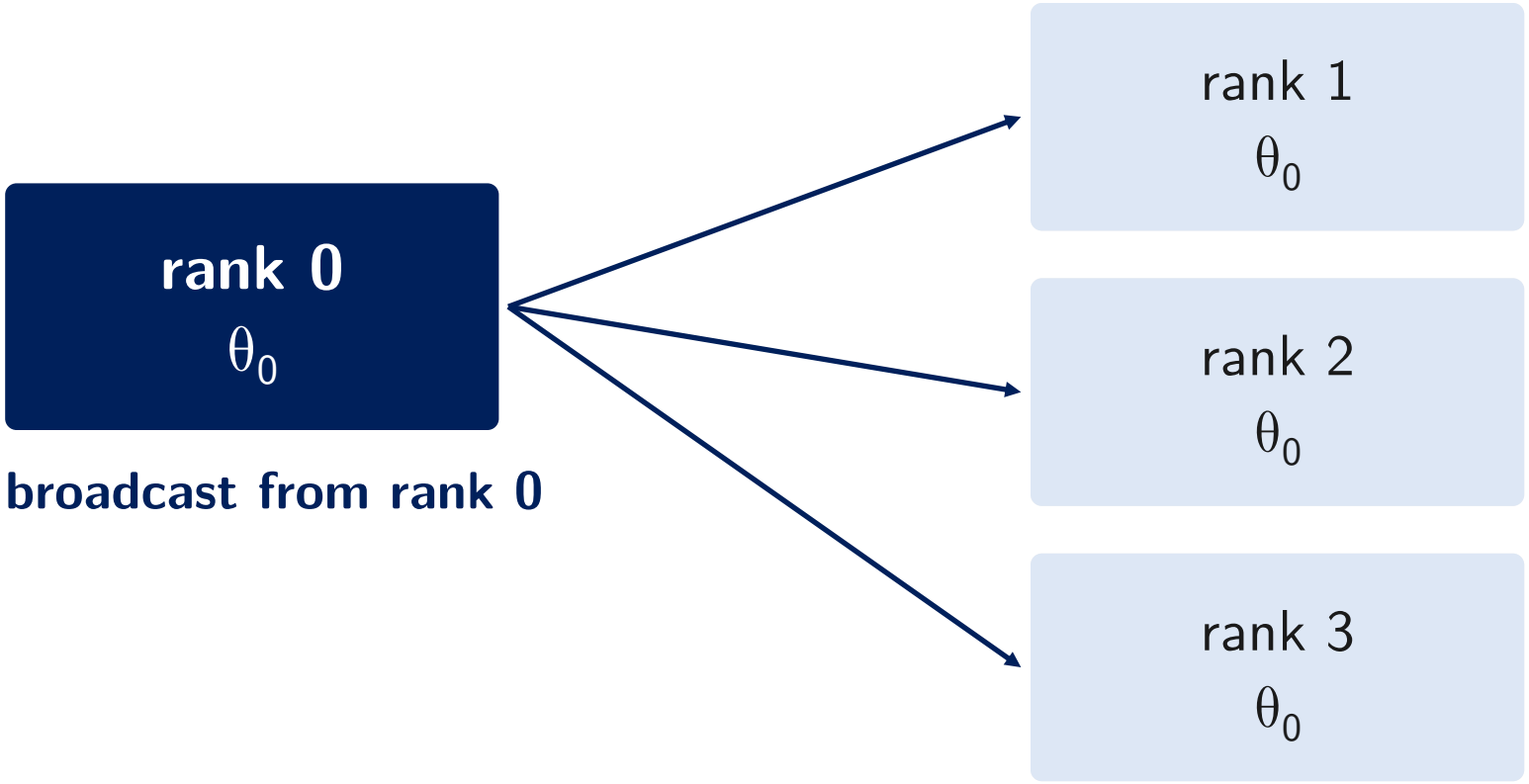
- Same loop as on one GPU.
- **backward()** also averages the gradients.

Key idea

DDP does three things: **start identical**, **bucket** the gradients, **overlap** the all-reduce with backward.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Start identical, stay identical



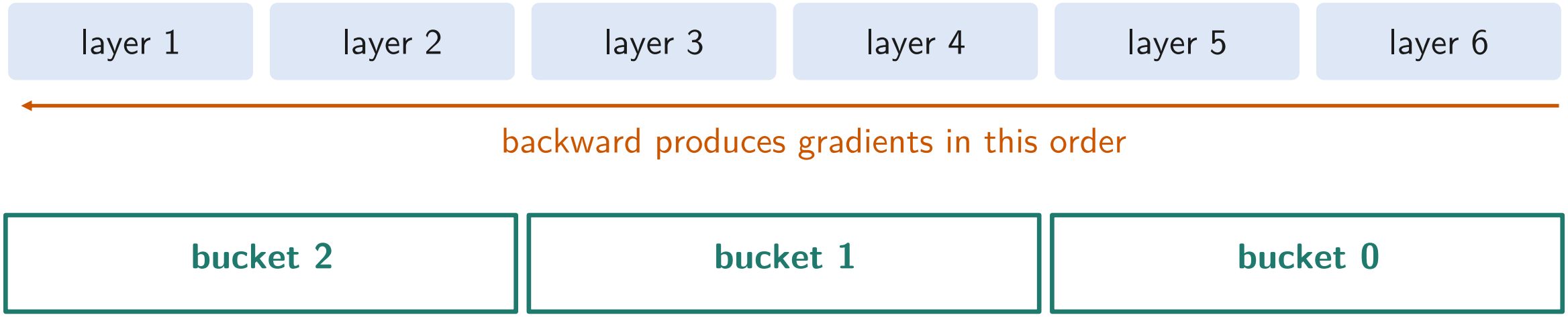
WHY IT LASTS

Same start + same averaged gradient + same optimizer $\Rightarrow$ the copies **stay identical**.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Buckets: fewer, larger all-reduces

a model's layers, input to output



one all-reduce per bucket, as soon as all of its gradients are ready

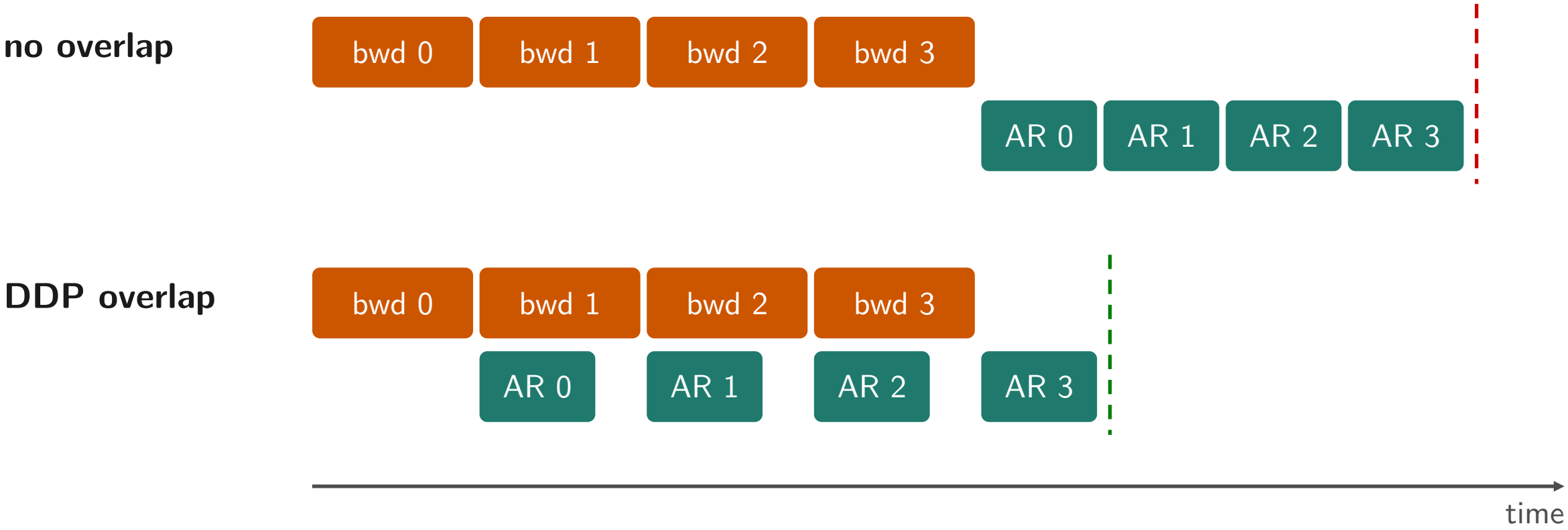
Memory

Fewer calls means fewer latency costs — and bucket 0 can be sent **while backward continues**.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Overlap: send while backward runs

bwd: backward of one bucket · AR: its all-reduce



Key idea

Step time $\approx$ max(backward, all-reduce) + **one bucket**, instead of the sum.

Check yourself

- Which term of the ring formula dominates for our MLP's gradient? For the 15 GB one?
- Why does DDP start by copying rank 0's parameters to every rank?
- Will DDP's overlap speed up our 34,338-parameter MLP? Why or why not?

Part 3

Where the memory goes

Adding GPUs divides the batch. Does it divide the memory?

Model states: 16 bytes per parameter

16-bit parameters	2Ψ
16-bit gradients	2Ψ
FP32 master parameters	4Ψ
Adam: average gradient	4Ψ
Adam: average squared gradient	4Ψ

optimizer state: 12Ψ

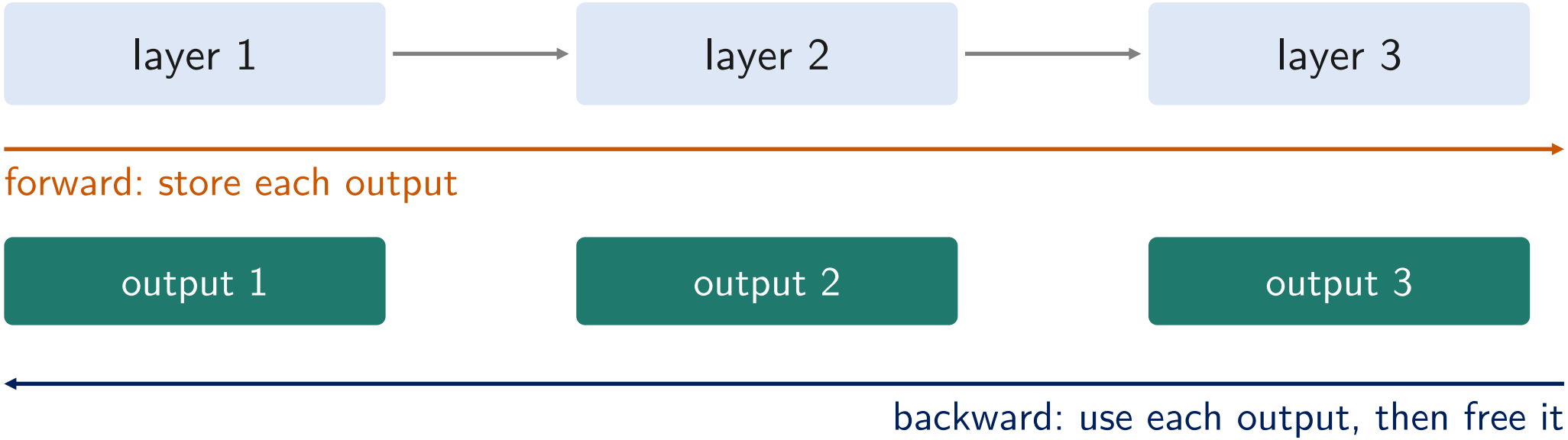
$$\text{total} = (2 + 2 + 12)\ \Psi = 16\Psi \text{ bytes}$$

$\Psi = 7.5 \times 10^9$: **120 GB**

our MLP: $\approx$ **0.55 MB**

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 3.1.

Activations: kept from forward for backward



Key idea

Activations come **per example**: twice the examples on a GPU, twice the activation memory.

Activations for our MLP

$$\text{one example: } 128 + 64 + 322 = 514 \text{ numbers}$$

$$\times\ 4 \text{ bytes (FP32)} = 2{,}056 \text{ bytes}$$

$$\times\ \mathcal{B} = 64\text{: } \approx 132 \text{ KB}$$

Memory

Our MLP: activations $\approx$ 132 KB, model states $\approx$ 0.55 MB. For large models, activations can be **just as big**.

Data parallelism copies the model states



blue: 16-bit params 2Ψ · orange: 16-bit grads 2Ψ · green: optimizer 12Ψ

Key idea

N identical copies of all 16Ψ bytes — the optimizer state alone is **75%**. Next lecture removes the copies.

What adding GPUs divides

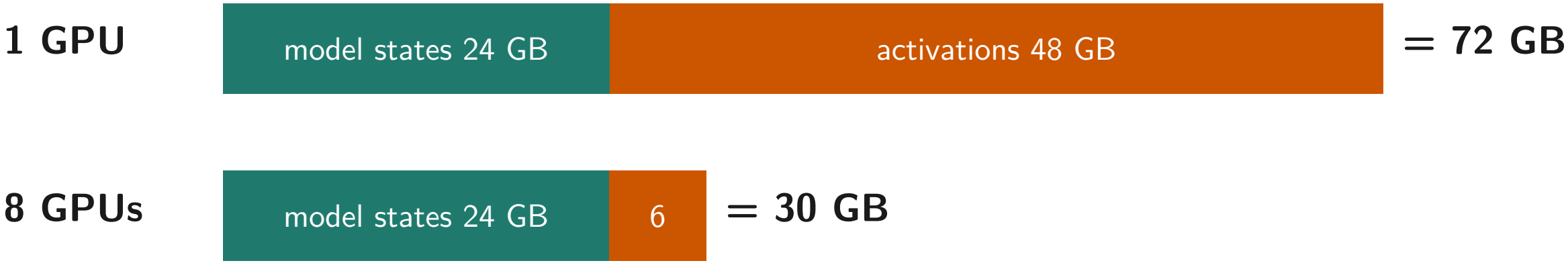
Memory	Size set by	With N GPUs, per GPU
Model states	the model: 16Ψ	16Ψ (**not** divided)
Activations	the examples	A / N (divided)

$$\text{per-GPU memory} = 16\Psi + A \;/\; N$$

Key idea

A: activations of the **whole** global batch. More GPUs shrink only the A/N term.

Back to the puzzle



Answer

The 24 GB of model states is on **every** GPU. Only the activations split, so 8 GPUs give 30 GB each.

When the copies do not fit

$$16\Psi \leq 80 \text{ GB} \quad \Rightarrow \quad \Psi \leq 5 \times 10^9 \text{ parameters}$$

- **Recompute** activations instead of storing them (less memory, more compute).
- **Shard** the model states across the GPUs: ZeRO and FSDP — **next lecture**.
- **Split** the model itself: tensor and pipeline parallelism — **Week 10**.

Watch out

Plain data parallelism cannot train a model whose $\mathbf{16\Psi}$ does not fit on one GPU — however many GPUs you add.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). (80 GB)

Check yourself

- List the five pieces of the 16 bytes per parameter.
- Our MLP, B = 64 on 4 GPUs: which part of each GPU's memory shrinks, and to what?
- What is the largest model plain data parallelism can train with Adam on 80 GB GPUs?

IN-CLASS BUILD

Check an update, count memory

Work in pairs. Part A is the core of T3; Part B is pencil and paper.

Exercise A: check a data-parallel update

```
xb, yb = shard(X, Y, rank, world)  # disjoint
F.cross_entropy(model(xb), yb).backward()
for p in model.parameters():
    dist.all_reduce(p.grad)        # SUM
    p.grad /= ____                 # (1)
F.cross_entropy(ref(X), Y).backward()  # 1 proc
torch.testing.assert_close(grads(model),
                           grads(ref))
```

QUESTIONS

(1) Fill in the blank.

(2) 17 examples on 2 ranks, 9 and 8: what changes?

(3) Use DDP instead. Does it still pass with 9 and 8?

model and ref start with the same weights; ref sees the whole batch on one process.

Exercise B: account for memory

- (1) Model states per GPU?
- (2) Per-GPU memory on **N = 8** GPUs?
- (3) The **fewest** GPUs (a power of 2) that fit?
- (4) $\Psi = 6 \times 10^9$ instead: how many GPUs do you need?

GIVEN

$\Psi = 3 \times 10^9$, Adam, mixed precision.

Activations: 1.5 GB per example.

Global batch B = 64.

GPUs of 80 GB. GB = 10^9 bytes.

$$\text{per-GPU memory} = 16\Psi + A \;/\; N, \quad A = \text{activations of the global batch}$$

Solutions A: the update check

- (1) `world` — the number of ranks (equal shards).
- (2) Each rank sends b[k] $\cdot$ g[k]; after the SUM, divide by B $= 17$.
- (3) DDP divides by N: it **fails** for $9 + 8$ unless you rescale each rank's loss.

In our pipeline

Passing on $8 + 8$ is not enough: T3 tests the **uneven** case.

Solutions B: the memory count

(1) $16\Psi = 48$ GB, on every GPU

(2) $48 + 96 \;/\; 8 = 60$ GB

(3) $\mathcal{N} = 4$: $48 + 96 \;/\; 4 = 72$ GB

(4) $16\Psi = 96$ GB > 80 GB: no $\mathcal{N}$ is enough

Check

More GPUs fix the **activations**, never the model states: (4) needs next lecture's sharding.

How to test it

- Run on **CPU** first (gloo backend), then on GPUs (NCCL).
- Compare **gradients and parameters** after the optimizer step.
- Use `assert_close` with its **default** tolerances.
- Test an **uneven** batch of 17.
- Delete the division: the test must **fail**.

In our pipeline

T3, part 1: hand-written DP vs a single-process reference vs DDP, on D3 data.

Key takeaways

- Gradient of a mean = mean of gradients: **all-reduce SUM, divide by N** — equal shards only; otherwise weight by **b[k]/B**.
- One all-reduce per step: **latency-bound** for our MLP, $\approx$ **2S**/β for big models.
- **DDP**: start identical, bucket the gradients, overlap the all-reduce with backward.
- Training with Adam: **16 bytes per parameter**, copied on every GPU by plain DP.
- Per-GPU memory = $\mathbf{16\Psi + A/N}$: more GPUs shrink only the activations.

Next time, and readings

When	What
Lecture 2 (this week)	Sharding the state: ZeRO and FSDP
This week	T2 checkpoint; Lab 4 due
Week 10, Lecture 1	Tensor parallelism: splitting one layer across GPUs

READING

Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020 — Sections 1 and 3. PyTorch Distributed Overview (DDP).